moustaches

Fench statistics (tables and diagrams)

v0.1.3 Akilon <https://github.com/Akilon27> MIT

About

While preparing my lessons, I realized that none of the existing packages for creating statistical diagrams support counts, the French definitions of quartiles, or the construction of histograms and box plots as I teach them… This package was created to fill that gap.

I took inspiration from parts of the LaTeX package ProfCollege (by Christophe Poulain), but the implementation is not a copy, and I did not use AI at all in the development of this package.

In the updates, I added proportionality tables, equation solving and matrix operations (with arrows for explanations and mblock for blocks spanning multiple columns/rows) and a few helper functions (with fletcher 0.5.8) and help-en (made by tidy 0.4.3).

Main module: “stat” and “gridmath” module

All the functions in this part from `math-grid` onward are part of the `gridmath` module which can be imported on its own.

- simpli()
- pgcd()
- bandes()
- camembert()
- histogramme()
- caracteristiques()
- stat()
- math-grid()
- lin-up()
- lin-down()
- row-op()
- x-product()
- col-op()
- col-combine()
- rdarrow()
- ruarrow()
- ldarrow()
- luarrow()
- urarrow()
- ularrow()
- drarrow()
- dlarrow()

simpli

Writes the quotient `a / b`, where `a` and `b` are floating-point numbers (or simply `a` if `b = 1`), together with its reduced fractional form (up to 6 decimal places).

```
#simpli(3.6,6.9)
```

$$\frac{3.6}{6.9} = \frac{12}{23}$$

Parameters

```
simpli(
  a: int float,
  b: int float
) -> content
```

pgcd

Computes the greatest common divisor (GCD) of a list of numbers.

```
#pgcd(1000, 1200, 1400, 1600, 1800,
2000)
```

```
200
```

Parameters

```
pgcd(..a: array) -> int
```

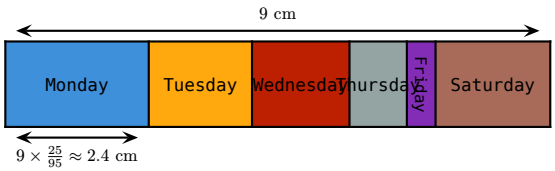
..a `array`

List of numbers

bandes

Draws a band chart.

```
#bandes(
  valeurs:("Monday", "Tuesday",
  "Wednesday", "Thursday", "Friday",
  "Saturday"),
  effectifs:(25, 18, 17, 10, 5, 20),
  width:9cm,
  tourne:(4,),
  explication:(0,),
)
```



Parameters

```
bandes(
  valeurs: array,
  effectifs: array,
  width: length,
  tourne: array,
  couleurs: auto array,
  print: boolean,
  explication: array,
  hauteur: length
) -> content
```

valeurs `array`

Categories (modalities)

Default: `()`

effectifs `array`

Counts of each category

Default: `()`

width `length`

Total width of the chart

Default: `1fr`

tourne `array`

Indices of labels to rotate by 90°

Default: `()`

couleurs `auto` or `array`

List of rectangle colors

Default: `auto`

print `boolean`

If `true`, replaces colors with tiling patterns

Default: `false`

explication `array`

Construction steps to explain, e.g. `(0,)` or `(0,3)`

Default: `()`

hauteur `length`

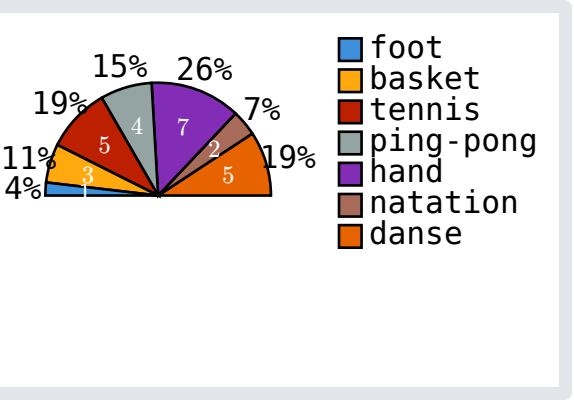
Height of the band chart

Default: `40pt`

camembert

Draws a (semi-)pie chart.

```
#camembert(
  valeurs:
  ("foot","basket","tennis","ping-
  pong","hand","natation","danse"),
  effectifs:(1, 3, 5, 4, 7, 2, 5),
  radius:1.5,
  legende:"d",
  espace:0,
  semi:true,
)
```



Parameters

```
camembert(
  valeurs: array,
  effectifs: array,
  data: array,
  semi: boolean,
  legende: str,
  espace: int float,
  print: boolean,
  couleurs: auto array,
  dedans: boolean none dictionary,
  dehors: boolean none dictionary,
  ..args: arguments
) -> content
```

valeurs `array`

Categories (modalities)

Default: `()`

effectifs `array`

Counts of each category

Default: `()`

data `array`

Allows passing a single data array instead

Default: `()`

semi `boolean`

If `true`, draws a semicircular chart

Default: `false`

legende `str`

Legend position: `"d"` (right), `"g"` (left), `"h"` (top), `"b"` (bottom), or `""` for none

Default: `"d"`

espace `int` or `float`

Spacing between legend items when the legend is on the left or right

Default: `.1`

print `boolean`

If `true`, replaces colors with tiling patterns

Default: `false`

couleurs `auto` or `array`

List of sector colors

Default: `auto`

dedans `boolean` or `none` or `dictionary`

`true`, `none`, `false`, or options passed to `inner-label`

Default: `true`

dehors `boolean` or `none` or `dictionary`

`true`, `none`, `false`, or options passed to `outer-label`

Default: `true`

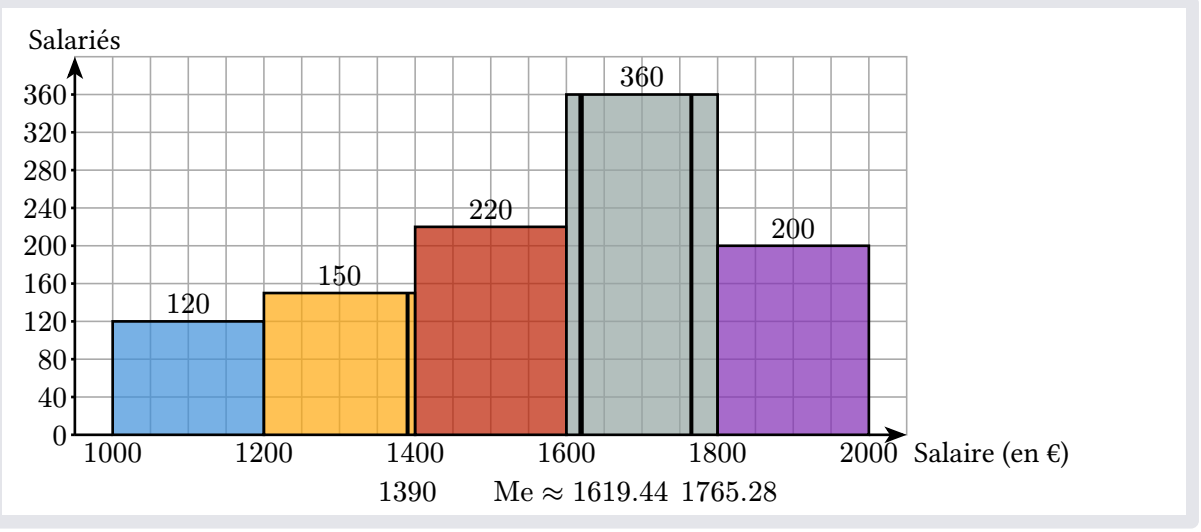
..args `arguments`

Other arguments to give to the piechart function of cetz

histogramme

Draws a true histogram: class widths do not have to be equal, and rectangle areas are proportional to the counts/frequencies.

```
#histogramme(
   classes: (1000, 1200, 1400, 1600, 1800, 2000),
   effectifs: (120, 150, 220, 360, 200),
   px: 50,
   uaire: 10,
   s: .5,
   // textsize: .5em,
   nom-donnee: [Salaire (en €)],
   nom-effectifs: [Salariés],
   mediane: true,
   quartiles: true,
   pos-rect: (0,5),
)
```



Parameters

```
histogramme(
  classes: array,
  effectifs: array,
  px: int,
  uaire: int,
  s: int float dictionary,
  grille: boolean,
  pas-grille: int float,
  nom-donnee: content,
  nom-effectifs: content,
  add: int array,
  lecture: boolean,
  donnees-sup: boolean,
  liste-couleurs: auto color array,
  pos-rect: array,
  print: boolean,
  transparence: ratio,
  ecc: boolean,
  mediane: boolean,
  quartiles: boolean,
  decimales: int,
  posq1: array,
  posmediane: array,
  posq3: array,
  stroke-mediane: length stroke,
  stroke-quartiles: length stroke,
  textsize: length
) -> content
```

classes `array`

Class boundaries: length = `effectifs.len() + 1`

Default: `()`

effectifs `array`

Counts of each class: length = `classes.len() - 1`

Default: `()`

px `int`

Tick interval on the x-axis

Default: 1

uaire `int`

Quantity represented by one grid square

Default: `1`

s `int` or `float` or `dictionary`

Scale passed to cetz

Default: `1`

grille `boolean`

Whether to display a grid

Default: `true`

pas-grille `int` or `float`

Grid spacing

Default: `1`

nom-donnee `content`

Label for the legend or y-axis, if applicable

Default: `[Values]`

nom-effectifs `content`

Label for the x-axis

Default: `[Frequencies]`

add `int` or `array`

Extra margin on the left, right, and bottom. Either a single integer or a triple

Default: `(-1, 1, 1)`

lecture `boolean`

Whether to display a y-axis when all classes have the same width

Default: `true`

donnees-sup `boolean`

Whether to display frequencies above the rectangles

Default: `true`

liste-couleurs `auto` or `color` or `array`

Rectangle colors, or a single color (e.g. `white` for printing)

Default: `auto`

pos-rect `array`

Position of the legend rectangle relative to the upper-left corner of the smallest rectangle

Default: `(0, 2)`

print `boolean`

If `true`, replaces colors with tiling patterns

Default: `false`

transparence `ratio`

Rectangle transparency. Use `100%` if `ListeCouleurs: white`

Default: `30%`

ecc `boolean`

Displays cumulative frequencies

Default: `false`

mediane `boolean`

Displays the median

Default: `false`

quartiles `boolean`

Displays the quartiles

Default: `false`

decimales `int`

Number of decimal places for rounded values

Default: `2`

posq1 `array`

Position of the Q1 label relative to the x-axis

Default: `(0,-1.5)`

posmediane `array`

Position of the median label relative to the x-axis

Default: `(0,-1.5)`

posq3 `array`

Position of the Q3 label relative to the x-axis

Default: `(1,-1.5)`

stroke-mediane `length` or `stroke`

Stroke used to draw the median

Default: `2pt`

stroke-quartiles `length` or `stroke`

Stroke used to draw the quartiles

Default: `1.5pt`

textsize `length`

Text size

Default: `1em`

caracteristiques

Computes descriptive statistics and returns them as a dictionary: `effectif-total`, `mediane`, `classe-mediane`, `q1`, `q3`, `d1`, `d9`, `ecart-interquartiles`, `max`, `min`, `etendue`, `modes`, `effectif-modes`, `sommex`, `sommex2`, `moyenne`, `variance`, `ecart-type`, `variance-echantillon`, `ecart-type-echantillon`, `lqboxplot`, and `cetzboxplot`.

```
#caracteristiques(sondage:(1, 5, 7, 9,
4, 5, 9, 8, 7, 3, 8))
```

```
(
  effectif-total: 11,
  mediane: 7,
  classe-médiane: none,
  q1: 4,
  q3: 8,
  d1: 3,
  d9: 9,
  ecart-interquartiles: 4,
  max: 9,
  min: 1,
  etendue: 8,
  modes: (5, 7, 8, 9),
  effectif-modes: 2,
  sommex: 66,
  sommex2: 464,
  moyenne: 6.0,
  variance: 6.18181818181818,
  ecart-type: 2.4863262420322436,
  variance-echantillon: 6.799999999999997,
  ecart-type-echantillon: 2.607680962081059,
  lqboxplot: (
    median: 7,
    q1: 4,
    q3: 8,
    whisker-low: 3,
    whisker-high: 9,
    outliers: (1,),
    mean: 6.0,
  ),
  cetzboxplot: (
    q1: 4,
    q2: 7,
    q3: 8,
    min: 3,
    max: 9,
    outliers: (1, 6.0),
  ),
)
```

Parameters

```
caracteristiques(
  valeurs: array,
  effectifs: array,
  classes: array,
  bins: int,
  sondage: array,
  crochets: boolean,
  deciles: boolean
) -> dictionary
```

valeurs `array`

Values of the numeric variable

Default: `()`

effectifs `array`

Counts of each value

Default: `()`

classes `array`

Class boundaries, if applicable: length = `effectifs.len() + 1`

Default: `()`

bins `int`

For raw data in `sondage`, automatically creates `bins` classes of equal width

Default: `0`

sondage `array`

List of all observations

Default: `()`

crochets `boolean`

For class intervals, use interval notation (`[a; b[`) instead of inequalities (`a <= ... < b`)

Default: `false`

deciles `boolean`

If `true`, whiskers are based on deciles; otherwise they extend to ±1.5 × IQR

Default: `true`

stat

Performs a statistical analysis and displays the results as a table, with optional frequencies (`effectifs`), relative frequencies (`"f"` for fractions, `"d"` for decimals, `"t"` for all, `true` for percentages), sector angles for (semi-)pie charts, cumulative frequencies (`ECC`, `FCC`), class intervals (automatically generated when `bins > 0`) and their centers.

It can also display charts: `"hbar"`, `"bar"`, `"circ"`, `"semicirc"`, `"box"`, `"bande"`, `"histo"`, or any combination of these.

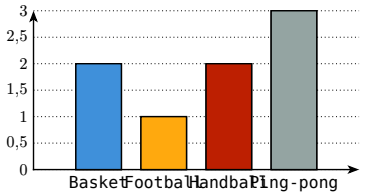
If `multi != ()`, grouped bar charts (horizontal or vertical) or multiple box plots can be displayed. Setting `print: true` replaces colors with tiling patterns.

```
#stat(
  classes:(150, 160, 170, 200),
  effectifs:(3, 4, 5),
  frequences:"f",
)
```

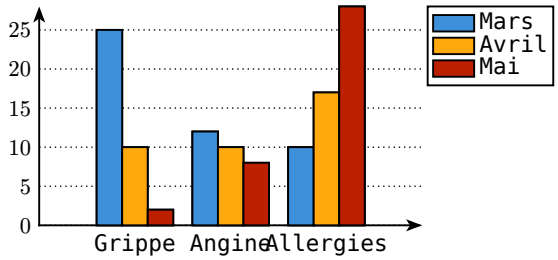
	$150 \leqslant ... < 160$	$160 \leqslant ... < 170$	$170 \leqslant ... < 200$
Centres des classes	155	165	185
Effectifs	3	4	5
Fréquences	$\frac{3}{12} = \frac{1}{4}$	$\frac{4}{12} = \frac{1}{3}$	$\frac{5}{12}$

```
#stat(
  sondage:
  ("Handball","Basket","Football","Handba
           "Ping-pong","Basket","Ping-
           pong","Ping-pong"),
  diagramme:"bar",
  cbar:(size:(8.2,4))
)
```

	Basket	Football	Handball	Ping-pong
Effectifs	2	1	2	3
Fréquences (en %)	25	13	25	38




```
#stat(
  valeurs:
  ("Grippe","Angine","Allergies"),
  multi:((25,12,10),(10,10,17),
  (2,8,28)),
  diagramme:"bar",
  labels:("Mars","Avril","Mai"),
  tableau:false
)
```



Parameters

```
stat(
  valeurs: array,
  effectifs: array,
  qualitatif: boolean,
  totaux: boolean,
  tableau: boolean,
  couleur-tableau: color,
  inset: length,
  nom-donnee: content,
  nom-effectifs: content,
  decimales: int,
  frequences: boolean str,
  sondage: array,
  vide: boolean,
  angle: boolean str,
  ecc: boolean str,
  classes: array,
  bins: int,
  crochets: boolean,
  centre: boolean str,
  colonnes-vide: array,
  cases-vide: array,
  diagramme: str,
  bar: array,
  cbar: array,
  moustaches: array,
  cmoustaches: array,
  circ: dictionary,
  bande: array,
  histo: array,
  fcc: boolean str,
  multi: array,
  labels: array,
  print: boolean,
  liste-couleurs: auto color array
) -> content
```

valeurs `array`

Values or categories of the variable

Default: `()`

effectifs `array`

Counts of each value

Default: `()`

qualitatif `boolean`

Whether the variable is qualitative

Default: `true`

totaux `boolean`

Display totals in tables

Default: `false`

tableau `boolean`

Display the table

Default: `true`

couleur-tableau `color`

Background color of the first row and first column

Default: `luma(85%)`

inset `length`

Cell inset

Default: `5pt`

nom-donnee `content`

Label for the upper-left table cell (and some charts)

Default: `[]`

nom-effectifs `content`

Label for the frequency row (and some charts)

Default: `[Frequencies]`

decimales `int`

Number of decimal places for rounded values

Default: `0`

frequences `boolean` or `str`

Relative frequencies: `true` for percentages, `"f"` for fractions, `"d"` for decimals, `"t"` for all, `"v"` for an empty row

Default: `true`

sondage `array`

List of all observations

Default: `()`

vide `boolean`

If `true`, all rows except the first are left blank

Default: `false`

angle `boolean` or `str`

Display sector angles (`"v"` for an empty row)

Default: `false`

ecc `boolean` or `str`

Display cumulative frequencies (`"v"` for an empty row)

Default: `false`

classes `array`

Class boundaries for continuous data: length = `effectifs.len() + 1`

Default: `()`

bins `int`

For raw data, automatically creates `bins` classes of equal width

Default: `0`

crochets `boolean`

Use interval notation (`[a; b[`) instead of inequalities (`a <= ... < b`)

Default: `false`

centre `boolean` or `str`

Display class centers (`"v"` for an empty row)

Default: `true`

colonnes-vide `array`

Columns to leave empty

Default: `()`

cases-vide `array`

Individual cells to leave empty, numbered row by row

Default: `()`

diagramme `str`

Chart type: `"hbar"`, `"bar"`, `"circ"`, `"semicirc"`, `"box"`, `"bande"`, `"histo"`, or any combination

Default: `""`

bar `array`

Parameters passed to the horizontal/vertical bar chart canvas

Default: `()`

cbar `array`

Parameters passed to cetz's `barchart` or `columnchart`

Default: `()`

moustaches `array`

Parameters passed to the custom box-plot function

Default: `()`

cmoustaches `array`

Parameters passed to cetz's `boxwhiskers`

Default: `()`

circ `dictionary`

Parameters passed to `camembert`

Default: `(:)`

bande `array`

Parameters passed to `bandes`

Default: `()`

histo `array`

Parameters passed to `histogramme`

Default: `()`

fcc `boolean` or `str`

Display cumulative relative frequencies (`"v"` for an empty row)

Default: `false`

multi `array`

Multiple series for grouped bar charts or multiple box plots

Default: `()`

labels `array`

Labels for the different series

Default: `()`

print `boolean`

If `true`, replaces colors with tiling patterns

Default: `false`

liste-couleurs `auto` or `color` or `array`

Colors used in the charts

Default: `auto`

math-grid

All the following functions are part of the “gridmath” module

Draws a **proportionality table** by generating named Fletcher nodes from an equation (with alignments or a matrix). By default, all nodes in the same column have the same width, and all nodes in the same row have the same height. In addition, all nodes are placed directly next to each other. Works like `diagram` (or mat) in math mode.

This function can also be used to solve equations and matrix (with arrows for explanations) with `equation:true` (see example in the README) or for matrix with `matrix-mode:true` or `matrix-mode:"[]"` etc.

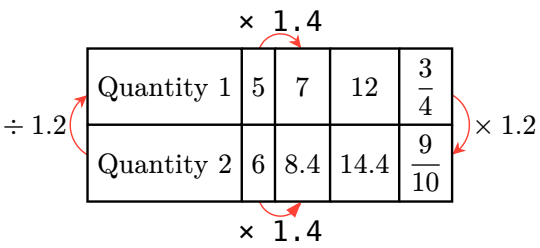
```
// Fletcher diagram with the same
parameters as the default values
// of the proportionnalite function
#diagram($
  "Quantity 1" & 5 & 7   & 9    & 3/4 \
  "Quantity 2" & 6 & 8.4 & 10.8 & 9/10
$,
  node-stroke: 1pt,
  node-shape: rect,
  spacing: 0cm,
  mark-scale: 1.2,
  node-inset: 6pt,)
```

Quantity 1	5	7	9	$\frac{3}{4}$
Quantity 2	6	8.4	10.8	$\frac{9}{10}$

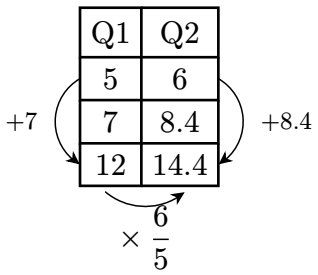
```
#math-grid($
  "Quantity 1" & 5 & 7   & 9    & 3/4 \
  "Quantity 2" & 6 & 8.4 & 10.8 & 9/10
$,
)
```

Quantity 1	5	7	9	$\frac{3}{4}$
Quantity 2	6	8.4	10.8	$\frac{9}{10}$

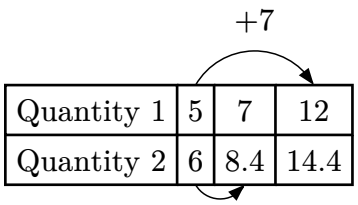
```
#math-grid(
  $ "Quantity 1"  & 5 edge("r", "-}>",
  bend: #70deg, label: times 1.4, label-
  sep: #(-1pt),shift:#(5pt)) & 7 & 12 &
  3/4  \
  "Quantity 2" & 6 edge("r", "-|>",
  bend: #(-70deg), label: times 1.4,
  label-sep: #(-1pt),shift:#(-5pt)) &
  8.4 & 14.4 & 9/10 $,
edge-stroke: .6pt+red, coef: $times
1.2$, coef-recip: $div 1.2$,
node-stroke: 1pt)
```



```
#math-grid($"Q1" & 5 & 7 & 12 \ "Q2" &
6 & 8.4 & 14.4$,
  // Change the orientation to a
  vertical table
  flip:true,
  // Arc (above) on the left connecting
  column 1 to column 3
  col-op(0, 1, 3, $+ 7$, flip:true),
  // Arc (below) on the right connecting
  column 1 to column 3
  col-op(1, 1, 3, $+ 8.4$, flip:true),
  coef: $times 6/5$,coef-args: (label-
  sep:2.5pt))
```




```
// Cells are automatically named <c-
column-row>
#math-grid(
  $
    "Quantity 1" & 5 & 7 & 12 \
    "Quantity 2" & 6 & 8.4 & 14.4
  $,
  // Connects the top of cell (1,0) to
  the top of cell (3,0)
  edge(<1-0.north>, <3-0.north>, "-|>",
  bend: 60deg, label: $+ 7$),
  // Connects the bottom-right of (1,1)
  to the bottom-left of (2,1)
  edge(<1-1.south>, <2-1.south>, "-|>",
  bend: -60deg),
)
```



Parameters

```
math-grid(
  eq: math.equation,
  equation: boolean,
  matrix-mode: boolean array str,
  matrix-sep: length,
  augment: none int array dictionary,
  display: boolean,
  equal-columns: boolean,
  equal-rows: boolean,
  node-stroke: none length color stroke,
  cadre: boolean,
  node-shape: function,
  node-align: auto alignment,
  inset: array,
  spacing: length,
  mark-scale: int float,
  flip: boolean,
  coef: none content,
  coef-recip: none content,
  coef-args: dictionary,
  coef-recip-args: dictionary,
  no-empty-node: boolean,
  fill-empty: boolean color,
  min-width: length,
  min-height: length,
  fill-map: dictionary function,
  header-row: boolean,
  header-col: boolean,
  header-fill: color,
  header-align: none alignment,
  clean-grid: boolean,
  ..args: arguments
) -> content
```

eq `math.equation`

Mathematical equation to be converted into a table, like in fletcher or as a matrix (to get the usual column-gap, it needs to be set to `0.2em`)

equation `boolean`

equation-mode to get the alternatingly right- and left-aligned columns, no stroke and no empty nodes

Default: `false`

matrix-mode `boolean` or `array` or `str`

matrix-mode to get the parenthesis or brackets, no lines etc if true, defaut to parenthesis, else one or more symbols can be given like for the mat function of typst.

Default: `false`

matrix-sep `length`

separation of the parens or brackets etc

Default: `0pt`

augment `none` or `int` or `array` or `dictionary`

draw augment lines in a matrix, like for the mat function of typst.

Default: `none`

display `boolean`

to get everything in display mode (big fractions etc)

Default: `true`

equal-columns `boolean`

Whether cells in a column should have the same width

Default: `true`

equal-rows `boolean`

Whether cells in a row should have the same width

Default: `true`

node-stroke `none` or `length` or `color` or `stroke`

Table line properties

Default: `1pt`

cadre `boolean`

to have an external border or not

Default: `true`

node-shape `function`

Node shape

Default: `rect`

node-align `auto` or `alignment`

alignment within nodes

Default: `auto`

inset `array`

Node inset (h,v) in points or with a couple of lengths

Default: `(4, 4)`

spacing `length`

Spacing between table cells

Default: `0cm`

mark-scale `int` or `float`

Scale of arrow marks

Default: `1.2`

flip `boolean`

Whether to make the table vertical

Default: `false`

coef `none` or `content`

Proportionality coefficient (from the 1st row to the 2nd)

Default: `none`

coef-recip `none` or `content`

Reciprocal coefficient (from the 2nd row to the 1st)

Default: `none`

coef-args `dictionary`

Optional arguments passed to the `edge` function for the coefficient

Default: `()`

coef-recip-args `dictionary`

Optional arguments passed to the `edge` function for the reciprocal coefficient

Default: `()`

no-empty-node `boolean`

to have the empty nodes drawn

Default: `true`

fill-empty `boolean` or `color`

to give a color to empty nodes

Default: `false`

min-width `length`

minimal width (useful for empty columns)

Default: `0pt`

min-height `length`

minimal height (useful for empty rows)

Default: `0pt`

fill-map `dictionary` or `function`

dictionaty “x-y” or “x,y”: color or function (x, y) => color to color cells

Default: `(:)`

header-row `boolean`

to apply the header-fill color to the first line (y = 0)

Default: `false`

header-col `boolean`

to apply the header-fill color to the first column (x = 0)

Default: `false`

header-fill `color`

color for the header

Default: `rgb("d0d0d0")`

header-align `none` or `alignment`

to apply a different alignment to the header (x = 0 ou y = 0)

Default: `none`

clean-grid `boolean`

clean the grid by making sure the lines aren't doubled with spacing = 0cm

Default: `true`

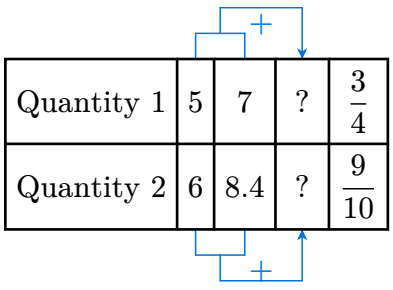
..args `arguments`

Optional arguments passed to the Fletcher `diagram` function

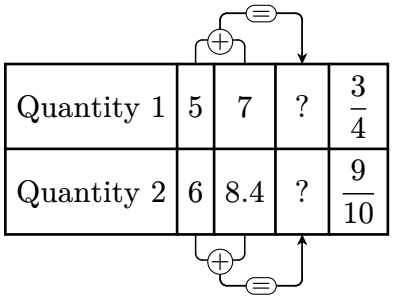
lin-up

Function showing that one column is the sum (default) or difference of two other columns, above the first row. Three modes are available.

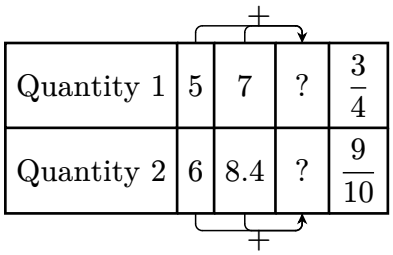
```
#math-grid(
  $ "Quantity 1"  & 5  & 7 & ? & 3/4  \
  "Quantity 2" & 6 & 8.4 & thick ? thick
  & 9/10 $,
  lin-up(1,2,3,sign: $ + $,color:
  blue,),
  lin-down(1,2,3,sign: $ + $, color:
  blue,),)
```



```
#math-grid(
  $ "Quantity 1"  & 5  & 7 & ? & 3/4  \
  "Quantity 2" & 6 & 8.4 & thick ? thick
  & 9/10 $,
  lin-up(1,2,3,sign: $ + $,mode:2,),
  lin-down(1,2,3,sign: $ + $, mode:2,),)
```




```
#math-grid(
  $ "Quantity 1"  & 5  & 7 & ? & 3/4  \
  "Quantity 2" & 6 & 8.4 & thick ? thick
  & 9/10 $,
  lin-up(1,2,3,sign: $ + $,mode:3,),
  lin-down(1,2,3,sign: $ + $, mode:3,),)
```



Parameters

```
lin-up(
  a: int,
  b: int,
  c: int,
  sign: content str,
  color: auto color,
  text-color: auto color,
  mode: int,
  height: length,
  ..args: dictionary
) -> content
```

a `int`

Index of the first starting column

b `int`

Index of the second starting column

c `int`

Index of the destination column

sign `content` or `str`

Operation sign

Default: `$ + $`

color `auto` or `color`

Arrow color

Default: `auto`

text-color `auto` or `color`

Text color

Default: `auto`

mode `int`

Three modes are available

Default: `1`

height `length`

“height” of the arrows in the 1st mode

Default: `7.24pt`

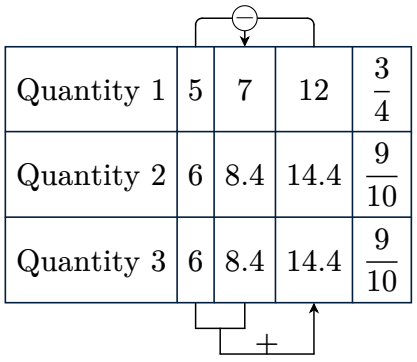
..args `dictionary`

Additional arguments for the edges

lin-down

Function showing that one column is the sum (default) or difference of two other columns, below the second row (or `n:5` if 5 rows). Three modes are available

```
#math-grid(
$"Quantity 1" & 5 & 7   & 12   & 3/4 \
 "Quantity 2" & 6 & 8.4 & 14.4 & 9/10 \
 "Quantity 3" & 6 & 8.4 & 14.4 & 9/10$,
   lin-up(1,3,2,sign: $ - $,mode:2),
   lin-down(1,2,3,n:3),
   node-stroke: 0.6pt+navy,
)
```



Parameters

```
lin-down(
  a: int,
  b: int,
  c: int,
  sign: content str,
  color: auto color,
  text-color: auto color,
  mode: int,
  n: int,
  height: length,
  ..args: dictionary
) -> content
```

a `int`

Index of the first starting column

b `int`

Index of the second starting column

c `int`

Index of the destination column

sign `content` or `str`

Operation sign

Default: `$ + $`

color `auto` or `color`

Arrow color

Default: `auto`

text-color `auto` or `color`

Text color

Default: `auto`

mode `int`

Three modes are available

Default: `1`

n `int`

number of rows

Default: `2`

height `length`

“height” of the arrows in the 1st mode

Default: `7.24pt`

..args `dictionary`

Additional arguments for the edges

row-op

Function to show operations on the rows (of a matrix or other) with “square” arrows

Parameters

```
row-op(
  a: int,
  b: none int,
  c: int,
  label: content str,
  color: auto color,
  text-color: auto color,
  n: int,
  width: length,
  ..args: dictionary
) -> content
```

a `int`

number of the first beginning line

b `none` or `int`

number of the second beginning line

Default: `0`

c `int`

number of the arrival line

label `content` or `str`

label of the operation

color `auto` or `color`

arrow color

Default: `auto`

text-color `auto` or `color`

text color

Default: `auto`

n `int`

number of columns

Default: `2`

width `length`

“width” of the arrows

Default: `3em`

..args `dictionary`

Additional arguments for the edges

x-product

Function to display the cross-multiplication arrows between two columns.

```
#math-grid(
  $ "Quantity 1"  & 5 & 7 & ? & 3/4  \
  "Quantity 2" & 6 & 8.4 & 14.4 & 9/10
  $,
  x-product(2,),)
```

Quantity 1	5	7	?	$\frac{3}{4}$
Quantity 2	6	8.4	14.4	$\frac{9}{10}$

Parameters

```
x-product(
  color: color,
  sep: float,
  left-shift: float,
  right-shift: float,
  flip: boolean,
  ..a: int arguments
) -> content
```

color `color`

Arrow color

Default: `luma(40%)`

sep `float`

Distance from the centers of the nodes

Default: `.2`

left-shift `float`

Offset of the left endpoints of the arrows, if needed

Default: `0`

right-shift `float`

Offset of the right endpoints of the arrows, if needed

Default: `0`

flip `boolean`

For use with vertical tables

Default: `false`

..a `int` or `arguments`

Arguments necessarily containing the index of the first column and optionally that of a second column (otherwise +1), as well as arguments passed to the Fletcher `edge` function

col-op

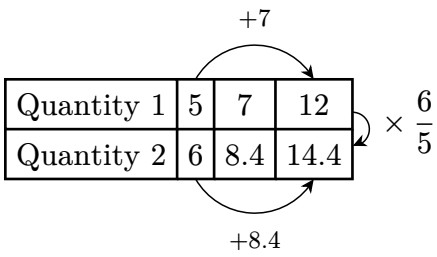
Draws a direct operation arrow from one column to another, above or below the table.

- `row`: row index (0 or 1)

- `f`: index of the starting column (e.g. 1)
- `t`: index of the destination column (e.g. 3)
- `label`: operation label (e.g. `$+ 7$` or `$+ "Col 2"$`)

and optionally:
- `pos`: `"top"` (above) or `"bottom"` (below)
- `bend`: arc curvature
- `flip`: indicates that the table is vertical
- additional arguments for the Fletcher `edge` command

```
#math-grid($"Quantity 1" & 5 & 7 & 12 \
"Quantity 2" & 6 & 8.4 & 14.4$,
  // Arc (au-dessus) à gauche liant la
  (colonne) ligne 1 et la (colonne)
  ligne 3
  col-op(0, 1, 3, $+ 7$),
  // Arc (en dessous) à droite liant la
  (colonne) ligne 1 et la (colonne)
  ligne 3
  col-op(1, 1, 3, $+ 8.4$),
  coef: $times 6/5$,coef-args: (label-
  sep:2.5pt,bend:90deg))
```



Parameters

```
col-op(
  row: int,
  f: int,
  t: int,
  label: content str,
  pos: auto str,
  bend: auto angle,
  flip: boolean,
  ..args: arguments dictionary
) -> content
```

row `int`

Index of the starting row (or column if flip:true)

f `int`

Index of the starting column (or row if flip:true)

t `int`

Index of the destination column (or row if flip:true)

label `content` or `str`

Text/operation (e.g. `$+ 12$` or `$+ "col 2"$`)

pos `auto` or `str`

Position: `"top"` (above) or `"bottom"` (below), default: auto

Default: `auto`

bend `auto` or `angle`

Arc curvature

Default: `auto`

flip `boolean`

For use with vertical tables

Default: `false`

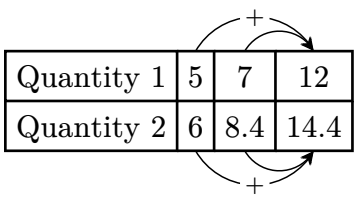
..args `arguments` or `dictionary`

Arguments passed to the Fletcher `edge` function

col-combine

Double arrow showing the combination (addition/subtraction) of two columns into a third.

```
#math-grid(
  $
    "Quantity 1" & 5 & 7 & 12 \
    "Quantity 2" & 6 & 8.4 & 14.4
  $,
  col-combine(0, 1, 2, 3, label: $+$,),
  col-combine(1, 1, 2, 3, label: $+$,),
)
```



Parameters

```
col-combine(
  row: int,
  col1: int,
  col2: int,
  to: int,
  label: content str,
  pos: auto str,
  bend: auto angle,
  flip: boolean,
  ..args: arguments dictionary
) -> content
```

row `int`

Index of the row (or column if flip:true)

col1 `int`

Index of the 1st starting column

col2 `int`

Index of the 2nd starting column

to `int`

Index of the destination column

label `content` or `str`

Text/operation (e.g. `$+ 12$` or `$+ "col 2"$`)

Default: `$+$`

pos `auto` or `str`

Position: `"top"` (above) or `"bottom"` (below), default: auto

Default: `auto`

bend `auto` or `angle`

Arc curvature

Default: `auto`

flip `boolean`

For use with vertical tables

Default: `false`

..args `arguments` or `dictionary`

Arguments passed to the Fletcher `edge` function

rdarrow

Right-side arrow going downward with a label

Parameters

```
rdarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved down

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

ruarrow

Right-side arrow going upward with a label

Parameters

```
ruarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved up

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

ldarrow

Left-side arrow going downward with a label

Parameters

```
ldarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved down

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

luarrow

Left-side arrow going upward with a label

Parameters

```
luarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved up

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

urarrow

Top-side (up) arrow going to the right with a label

Parameters

```
urarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved down

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

ularrow

Top-side (up) arrow going to the left with a label

Parameters

```
ularrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved down

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

drarrow

Bottom-side (down) arrow going to the right with a label

Parameters

```
drarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved down

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

dlarrow

Bottom-side (down) arrow going to the left with a label

Parameters

```
dlarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

arrow label

n `int`

number of rows moved down

Default: 1

..a `arguments` or `dictionary`

arguments passed to Fletcher's edge function

Sub-module for tilings and colormaps: “hachures”

- nelines()
- grille()
- nwlines()
- alveoles()
- etoile()
- stripes()
- dots()
- cross()

Variables

- hachures
- typst18
- cycles-couleurs

nelines

North-east lines tiling

Parameters

```
nelines(
  c: color,
  size: length,
  fill: color
) -> tiling
```

grille

grid lines tiling

Parameters

```
grille(
  c: color,
  size: length,
  fill: color
) -> tiling
```

nwlines

North-west lines tiling

Parameters

```
nwlines(
  c: color,
  size: length,
  fill: color
) -> tiling
```

alveoles
hexagons tiling

Parameters

```
alveoles(
  c: color,
  size: length,
  background: color,
  fill: none color
) -> tiling
```

etoile
stars tiling

Parameters

```
etoile(
  c: color,
  size: length,
  background: color,
  fill: none color
) -> tiling
```

stripes
colored stripes tiling that print in b&w

Parameters

```
stripes(
  c: color,
  size: length,
  fill: auto color
) -> tiling
```

dots
colored dots tiling that print in b&w

Parameters

```
dots(
  c: color,
  size: length,
  fill: auto color
) -> tiling
```

cross

colored cross tiling that print in b&w

Parameters

```
cross(
  c: color,
  size: length,
  fill: auto color
) -> tiling
```

hachures `array`

List of tilings

typst18 `array`

colormap of basic typst colors

cycles-couleurs `array`

25 cycles de couleurs disponibles